Software for Systematics and Evolution

Systematic Biology (2026), **0**, 0, *pp.* 1–??
doi:10.1093/sysbio/treeflow

TreeFlow: Probabilistic Modelling and Automatic Differentiation for Phylogenetics

Christiaan Swanepoel,[*,1,2]
Mathieu Fourment,[3]
Xiang Ji,[4]
Hassan Nasif,[5]
Marc A Suchard,[6,7,8]
Frederick A Matsen IV,[5,9,10]
Alexei J Drummond[1,11]

[1]Centre for Computational Evolution, The University of Auckland, Auckland, New Zealand;
[2]School of Computer Science, The University of Auckland, Auckland, New Zealand, 1010;
[3]Australian Institute for Microbiology and Infection, University of Technology Sydney, Ultimo NSW, Australia;
[4]Department of Mathematics, Tulane University, New Orleans, USA;
[5]Public Health Sciences Division, Fred Hutchinson Cancer Research Center, Seattle, Washington, USA;
[6]Department of Human Genetics, University of California, Los Angeles, USA;
[7]Department of Computational Medicine, University of California, Los Angeles, USA;
[8]Department of Biostatistics, University of California, Los Angeles, USA;
[9]Department of Statistics, University of Washington, Seattle, USA;
[10]Department of Genome Sciences, University of Washington, Seattle, USA;
[11]School of Biological Sciences, The University of Auckland, Auckland, New Zealand, 1010;
Correspondence to be sent to: School of Computer Science, The University of Auckland, Auckland, New Zealand, 1010;

Email: cswa648@aucklanduni.ac.nz

Abstract

Probabilistic modelling frameworks are powerful tools for statistical modelling and inference. They are not immediately generalisable to phylogenetic problems due to the particular computational properties of the phylogenetic tree object. TreeFlow is a software library for probabilistic modelling and automatic differentiation with phylogenetic trees. It embeds phylogenetic trees in the TensorFlow Probability framework, and implements

inference algorithms for phylogenetic models given a fixed tree topology. We demonstrate how TreeFlow can be used to quickly implement and assess new models. We also show that it provides reasonable performance for gradient-based inference algorithms compared to specialized computational libraries for phylogenetics.

Key words: phylogenetics; Bayesian inference; probabilistic modelling; variational inference

Introduction

Traditionally, Bayesian phylogenetic analyses have been performed by specialized software (Huelsenbeck and Ronquist 2001; Lartillot et al. 2009; Drummond et al. 2012). A number of software packages exist that implement a broad but predefined collection of models and associated specialized inference methods. Typically, inference is handled by carefully crafted but computationally costly Markov chain Monte Carlo (MCMC) methods (Metropolis et al. 1953; Hastings 1970). Considerable effort has been invested in improving the computational efficiency of MCMC for phylogenetics (Mateiu and Rannala 2006; Lakner et al. 2008; Höhna and Drummond 2012; Zhang and Drummond 2020). In contrast, in other realms of statistical analysis, *probabilistic graphical modelling* or *probabilistic programming* software libraries have entered into widespread use. These allow the specification of almost any model as a probabilistic program, and inference is provided automatically with a generic inference method. Exploiting the power of these tools in phylogenetic analyses could significantly accelerate research by making the process of developing new models and implementing inference faster and more flexible, as evidenced by recent work attempting to reconcile probabilistic modelling and programming frameworks with phylogenetic models (Fourment and Darling 2019; Ronquist et al. 2021).

Probabilistic modelling tools, such as BUGS (Lunn et al. 2000), Stan (Carpenter et al. 2017), PyMC3 (Salvatier et al. 2016), and TensorFlow Probability (Dillon et al.

2017) allow users to specify probabilistic models by describing the structure of a *probabilistic graphical model* with code. Typically, this involves composing pre-implemented probability distributions by specifying conditional dependencies between variables. This structure can be used to evaluate a joint probability density function for the model variables. Advancements in automatic inference methods have allowed these tools to perform efficient inference on a wide range of models. Some notable examples, including automatic differentiation variational inference (Kucukelbir et al. 2017) (ADVI) and Hamiltonian Monte Carlo (Duane et al. 1987) (HMC), use local gradient information from the model's probability density function to efficiently navigate the parameter space.

Another class of methods for performing statistical inference are *probabilistic programming languages*. These provide more flexibility than graphical modelling methods by explicitly describing the generative process of the model as a *probabilistic program*, rather than just composing probability distributions, and typically include inference schemes that do not require explicit evaluation of a joint probability density. Since the term *probabilistic programming* has been widely adopted to describe probabilistic graphical modelling software, probabilistic programming languages are often qualified as *universal* in that they are not restricted to composing pre-implemented probability distributions. Examples of universal probabilistic programming languages include Church (Goodman et al. 2008) and Pyro (Bingham et al. 2019).

One key technology that enables gradient-based automatic inference with probabilistic models is *automatic differentiation*. Automatic differentiation refers to methods which efficiently calculate machine-precision gradients of functions, specified by computer programs, without extra analytical derivation or excessive computational overhead compared to the function evaluation. Numerical programs consist of elementary arithmetic operations, and automatic differentiation tools apply the chain rule to these to compute derivatives directly, without requiring the construction of explicit mathematical expressions. Automatic differentiation frameworks that have statistical inference packages

built on top of them include Theano (Bergstra et al. 2010), Pytorch (Paszke et al. 2019), JAX (Bradbury et al. 2018) and TensorFlow (Abadi et al. 2016). Some of these extend to non-trivial computational constructs such as control flow and recursion (Yu et al. 2018).

The structure of the phylogenetic tree object is a major barrier to implementing efficient inference for phylogenetic probabilistic models. It is not clear how the association between its discrete and continuous quantities (the topology and branch lengths respectively) should be represented and handled in inference. Also, the combinatorial explosion of the size of the discrete part of the phylogenetic state space presents a major challenge to any inference algorithm. Generic random search methods for discrete variables, as in the naive implementation of MCMC sampling, do not scale appropriately to allow inference on modern datasets with thousands of taxa.

Efforts have already been made to apply probabilistic graphical modelling and probabilistic programming to phylogenetic methods (Höhna et al. 2014; Fourment and Darling 2019; Ronquist et al. 2021; Drummond et al. 2023). RevBayes (Höhna et al. 2016) implements a probabilistic modelling language that is focused on phylogenetic models. However, it requires the user to manually specify an MCMC scheme for performing inference. Universal probabilistic programming languages have also been used to express generative processes for trees and to automatically generate Sequential Monte Carlo inference schemes for complex speciation models. This line of work has further proposed how such languages might generate inference schemes for other phylogenetic models, including MCMC samplers over tree topologies (Ronquist et al. 2021; Senderov et al. 2024). Blang (Bouchard-Côté et al. 2022) is a probabilistic modelling framework which supports arbitrary data structures that has been used to implement phylogenetic models and can automatically construct inference schemes. Another tool, LinguaPhylo, provides a domain specific graphical modelling language for phylogenetics and has the capability to automatically generate a specification for an MCMC sampler for inference (Drummond et al. 2023).

Applying gradient-based inference methods to phylogenetics is a major challenge. *Probabilistic path Hamiltonian Monte Carlo* has been developed to use gradient information to sample phylogenetic posteriors across multiple tree topologies (Dinh et al. 2017), though moves between tree topologies are not totally informed by gradient information and are similar to the random-walk proposal distributions available to standard MCMC routines. Hamiltonian Monte Carlo proposals for continuous parameters within tree topologies has been paired with standard MCMC moves between topologies for estimating local mutation rates and divergence times using scalable algorithms for gradient calculation (Fisher et al. 2023; Ji et al. 2023). Another approach has used Stan to apply automatic differentiation variational inference to phylogenetic inference with a fixed rooted tree topology (Fourment and Darling 2019). Finally, variational inference has been applied to phylogenetic tree inference using a clade-based distribution on unrooted tree topologies (Zhang and Matsen IV 2018b). This approach has been extended to a more expressive family of approximating distributions (Zhang 2020), and applied to rooted phylogenies (Zhang and IV 2024). Variational inference has also been combined with combinatorial sequential Monte Carlo to approximate tree topologies and branch lengths jointly without relying on a predefined set of clade supports (Koptagel et al. 2022).

TreeFlow aims to enable the application of gradient-based inference methods to phylogenetic models. It implements a representation of phylogenetic trees in the TensorFlow computational framework which supports automatic differentiation, and a range of associated computations typically used in phylogenetic models of sequence evolution. These are all integrated with the TensorFlow Probability probabilistic modelling framework. TreeFlow provides utilities for applying variational Bayesian inference to phylogenetic models, particularly with a fixed tree topology, and a command line user interface.

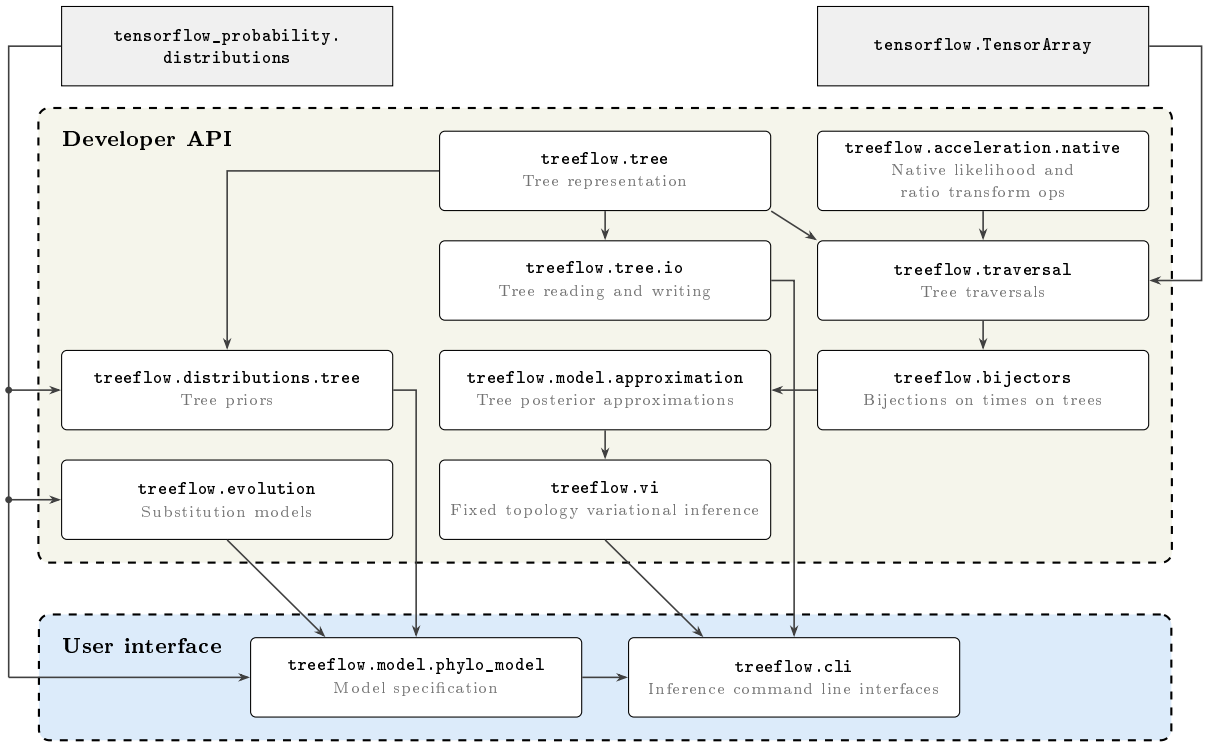


Fig. 1: Diagram showing the architecture of the TreeFlow package. Nodes represent Python modules and the arrows between them denote major dependencies. The dashed rectangles separate the components based on their primary interfaces. Nodes outside the two interface groups denote major external dependencies.

Software Description

TreeFlow is a Python library which provides modular tools for phylogenetic modelling and inference. It provides two primary interfaces: a user interface for performing parameter inference in fixed-topology models of a standard form, and a Python developer API for implementing and composing custom phylogenetic models. TreeFlow supports both rooted and unrooted phylogenetic trees, but the user interface is focused on inference with rooted trees. The architecture of the TreeFlow package is displayed in Figure 1.

We first describe the user interface, which provides the most accessible entry point for users wanting to apply TreeFlow to standard phylogenetic analyses. We then describe the developer API, which provides the modular components that underpin the user interface and allow users to implement new models.

User interface

TreeFlow provides command line interfaces for fixed-topology inference. These allow inference on standard phylogenetic models such as those performed by specialized software. For inputs, they take a nucleotide sequence alignment in the FASTA or Nexus format, a rooted tree topology in Newick format, and a TreeFlow model specification in YAML format. Command line interfaces are provided for both automatic differentiation variational inference and maximum a posteriori inference.

Because many phylogenetic analyses use models of a similar structure, TreeFlow provides a format for concisely specifying this kind of model. A single unpartitioned multiple sequence alignment can be modelled with a single substitution model, tree prior, and associated parameter priors. The model specification is a nested dictionary, with keys at various levels corresponding to model components, their parameters, and distributions used as priors. This is transformed into a TensorFlow Probability joint distribution. The model definition structure can be serialized in the YAML markup format which provides a text-based model definition format. An example of this is shown in Supplementary Appendix Figure S1. The substitution and tree models can be drawn from the models described below, as well as parameter priors from a range of standard probability distributions implemented in TensorFlow Probability. This model definition format is not intended to cover the full range of probabilistic models that can be implemented with TreeFlow's Python API, but rather provide a convenient input for TreeFlow's command line interfaces. Installation instructions, a complete reference for the YAML model definition format and its available model components, and a step-by-step tutorial are provided in the online manual (`https://treeflow.readthedocs.io`); new users may find the "rates and dates" tutorial, which works through a full analysis using this format, a helpful starting point after installing the software.

Developer API

The developer API allows users to implement new models and compose them with existing model components in a flexible structure, while leveraging the inferential machinery provided by TreeFlow. The TreeFlow Python library includes modules for the representation, traversal, and reading and writing of phylogenetic trees. There are modules implementing bijections for phylogenetic trees, phylogenetic posterior approximations, and fixed-topology variational Bayesian inference. It also implements tree priors and models of sequence evolution. TensorFlow Probability distributions can be composed into a *probabilistic graphical model* using TensorFlow Probability's joint distribution functionality (Piponi et al. 2020). The code to specify a joint distribution provides a concise representation of the model used in a data analysis. The ability to implement phylogenetic models in this framework means that automatic inference algorithms implemented in TensorFlow can be leveraged, as well as other utilities such as sampling from the joint prior. The subsections below describe the major components of the developer API.

Tree representation

TreeFlow is built on TensorFlow, a computational framework for machine learning. TreeFlow leverages TensorFlow's capabilities for accelerated numerical computation and automatic differentiation. TensorFlow operations can be executed on either the CPU or GPU, and as a result TreeFlow can make use of either type of processor. Many of TensorFlow's CPU operations are parallelized, allowing TreeFlow to make use of multiple processor cores. All of the benchmarks and examples in this paper, and TreeFlow's command line interfaces, use the CPU for computation.

TensorFlow's core computational object is the Tensor, a multi-dimensional array with a uniform data type. Phylogenetic trees are not immediately at home in a Tensor-centric universe, as they are a complex data structure, often defined recursively, with both continuous and discrete components. TreeFlow represents trees as a structure of

Tensors. This structure contains floating point Tensors representing the times and branch lengths, and integer Tensors representing the topology, all encapsulated in a Python object. The topological Tensors include indices of node parents, children, and for pre- and post-order traversals. TensorFlow has extensive support for structures of Tensors such as the TreeFlow tree class, including using them as arguments to compiled functions and defining probability distributions over complex objects. This means computations and models involving phylogenetic trees can be expressed naturally.

Tree distributions

TreeFlow uses the existing probabilistic modelling infrastructure provided by TensorFlow Probability, which implements standard statistical distributions and inferential machinery (Dillon et al. 2017). The Distribution interface encapsulates functionality for sampling from a distribution and computing its probability density or mass function. TreeFlow implements a tree distribution base class which allows TensorFlow Probability Distributions to be defined on TreeFlow's phylogenetic tree representation.

TreeFlow implements Distributions for some commonly-used generative models for phylogenetic trees. The models currently implemented, Kingman's coalescent (Kuhner et al. 1995) and the Birth-Death-Sampling speciation process (Stadler 2009), can be used as *tree priors* in phylogenetic probabilistic models and to infer parameters related to population dynamics from genetic sequence data. Since sampling from these distributions is non-trivial and not required for variational inference, only the probability densities of these models are currently implemented.

Models of sequence evolution

The models of nucleotide sequence evolution currently implemented in TreeFlow are the Jukes-Cantor (Jukes and Cantor 1969), HKY85 (Hasegawa et al. 1985), and General Time Reversible (GTR) (Tavaré 1986) models. TreeFlow includes a standard approach for

dealing with heterogeneity in mutation rate across sites based on marginalizing over a discretized site rate distribution (Yang 1994). The probabilistic modelling framework, however, allows for the use of any appropriate distribution as the base site rate distribution rather than just the standard single-parameter Gamma distribution. For example, it is straightforward to replace the base Gamma distribution with a Weibull distribution, which has a quantile function that is much easier to compute (Fourment and Darling 2019). Thanks to TensorFlow's vectorized arithmetic operations, it is also natural to model variations in mutation rate over lineages by specifying parameters for multiple rates (possibly with a hierarchical prior) and multiplying by the branch lengths of the phylogenetic tree. Models which can be naturally expressed this way include the log-Normal random local clock (Drummond et al. 2006), which is implemented with TreeFlow's model definition format, and auto-correlated relaxed clock models (Thorne et al. 1998), which would be straightforward to implement with the traversal module in the Python API.

Tree traversals

To perform model computations involving the phylogenetic tree, it is necessary to traverse the tree according to the structure specified by its topology. Since this structure is less accessible in an array-based tree representation, and native Python control flow cannot be used when the topology of the tree is a dynamic variable, TreeFlow provides utilities to perform tree traversals. These are used to efficiently implement some core computations and also made accessible to developers for extending TreeFlow's functionality.

When iterating over the nodes of the tree in an automatic differentiation framework it is important to consider computational complexity. Since modern genomic datasets can result in the number of nodes in the tree being extremely large, it is crucial to maintain linear scaling with the size of the tree. Any computation that produces values for every node and requires access to results for multiple previous nodes in the traversal might incur

a quadratic computational cost. This could occur when the topology is a dynamic variable as, at a given step of the iteration, values from any element of the Tensor representation might need to be accessed. Since TensorFlow's automatic differentiation framework depends on the immutability of Tensors, an intermediate representation of the state for the whole tree needs to be maintained for every step of the iteration, leading to a quadratic complexity. Intuitively, because a Tensor cannot be modified in place, a step of the traversal that updates the value at one node cannot simply overwrite that entry; it must instead produce a whole new copy of the tree-wide state Tensor. Repeating this for each of the roughly N nodes creates N copies each of size N, and differentiating through all of them, so both the computation and memory grow with the square of the number of nodes.

TreeFlow's solution to this scaling issue is to implement tree traversals using TensorFlow's `TensorArray` construct (Yu et al. 2018). The TensorArray is a data structure representing a collection of tensors. The TensorArray relaxes the requirement of immutability to a *write-once* property enforced on its constituent tensors. The TensorArray tracks dependencies between values so automatic differentiation can appropriately propagate gradients back through computation in linear time.

A notable computation that makes use of the tree traversal framework described above is the *phylogenetic likelihood*, the probability of a sequence alignment given a phylogeny and model of sequence evolution. This typically involves integrating out character states at unsampled ancestral nodes using a *dynamic programming* computation known as *Felsenstein's pruning algorithm* (Felsenstein 1981). Considerable attention has been given to the problem of efficiently computing gradients of branch lengths with respect to the phylogenetic likelihood (Ji et al. 2020). TreeFlow implements the pruning algorithm as a TensorArray-based *postorder* traversal. The recursion is expressed in terms of the per-branch transition probability matrices rather than directly in terms of branch lengths and substitution model parameters. This is a lower-level interface than that of specialised libraries such as BEAGLE, which take branch lengths and substitution parameters as

inputs and return the likelihood together with analytically derived gradients. Working at the level of transition probabilities is a more natural fit for automatic differentiation, because the gradient of the likelihood propagates back through the transition-probability computation to whatever parameters produced it. Consequently the branch-length gradient, and gradients with respect to any substitution model parameters, are obtained automatically without a hand-derived gradient expression for each model, and the branch-length gradient demonstrates linear scaling with respect to the number of taxa in our benchmarks (see the Benchmarks section below).

Because this pure-TensorFlow traversal obtains its gradients automatically and maintains linear scaling, it makes implementing a new model straightforward: a developer can compose the traversal with new model components and rely on automatic differentiation for the gradients, without writing native C++ code or deriving gradient expressions by hand. Where maximum performance is required, TreeFlow additionally provides a compiled C++ implementation of the pruning likelihood as a TensorFlow custom operation. This native operation performs the same recursion in native code, parallelised across alignment sites, and is paired with a hand-derived analytic gradient that reuses the partial likelihoods computed during the forward pass. It is registered with TensorFlow's automatic differentiation and selected by a single flag, so it is a drop-in accelerated alternative to the TensorArray traversal; its only additional requirement is a compilation step for the target platform. The two implementations are complementary: the portable traversal is convenient for model development, for computations with dynamic topologies, and for deployment without a build step, while the native operation supplies the performance needed for large-scale inference (see the Benchmarks section). We refer to them in the benchmarks as TreeFlow and TreeFlow (native) respectively.

Another useful tool for phylogenetic inference implemented in TreeFlow is the *node height ratio transform* (Kishino et al. 2001). This has been used to infer times on phylogenetic trees using maximum likelihood in the PAML software package (Yang 2007).

The ratio transform parametrizes the internal node heights of a tree as the ratio between a node's height and its parent's height. This has been applied to Bayesian phylogenetic inference in the context of automatic differentiation variational inference (Fourment and Darling 2019) and Hamiltonian Monte Carlo (Ji et al. 2023). The computation of node heights from ratios is implemented in TreeFlow as a TensorArray-based *preorder* traversal. As with the phylogenetic likelihood, this transform is also provided as an optional compiled C++ TensorFlow custom operation, paired with a hand-derived analytic gradient, for use when performance is critical; the pure-TensorFlow traversal remains the default and requires no compilation step.

Bijectors on phylogenetic trees

The ratio transform described above is wrapped into a TensorFlow Probability Bijector which provides an interface for transforming real-valued distributions into phylogenetic tree distributions. The Bijector is an interface which represents a transformation of samples from a multivariate probability distribution. It requires implementation of the forward transformation and its inverse (which necessarily exists if the transformation is a bijection), as well as the determinant of the Jacobian matrix of the transformation. This can be used by TensorFlow Probability to perform change of variable, rewriting the probability density of the base distribution in the space the bijector transforms it to.

The ratio transformation has a triangular Jacobian matrix, which means computing the determinant required for the change of variable formula can be computed in linear time with respect to the number of internal node heights (Fourment and Darling 2019). Using change of variable, a multivariate distribution on ratios can be transformed into a distribution on the internal node heights of a rooted phylogeny. Other TensorFlow Probability Bijectors can be used to transform real-valued distributions into ratios. Another Bijector implemented in TreeFlow combines node heights with a fixed tree

topology to produce a complete phylogenetic tree object. These bijectors can be composed sequentially, for example transforming unconstrained real values to ratios, then to node heights, then to a full tree, yielding a probability distribution over branch lengths for a given topology. The result can be used as an approximation to a phylogenetic posterior distribution and combined with models such as TreeFlow's tree priors and phylogenetic likelihood.

Variational inference

One form of statistical inference for which gradient computation is essential is variational Bayesian inference (Jordan et al. 1999). The goal of Bayesian inference is to characterise a *posterior distribution* which represents uncertainty over model parameters. Variational Bayesian inference achieves this by optimizing an approximation to the posterior distribution which has more convenient analytical properties. Typically, the optimisation routine used in variational inference can scale to a larger number of model parameters than the random-walk sampling methods used by MCMC methods.

One concrete implementation of variational inference is *automatic differentiation variational inference* (ADVI) (Kucukelbir et al. 2017). ADVI can perform inference on a wide range of probabilistic graphical models composed of continuous variables. It automatically constructs an approximation to the posterior by transforming a convenient base distribution to respect the domains of the model's component distributions. It then optimizes this approximation using stochastic gradient methods (Robbins and Monro 1951; Bottou 2010). Typically, independent Normal distributions are used as the base distribution for computational convenience. This is known as the *mean field approximation*, and for posterior distributions that have significant correlation-structure, skew, multi-modality, or heavy tails, can introduce error in parameter estimation (Blei et al. 2017). Possible solutions to this approximation error include highly flexible variational approximations with large numbers of parameters (Rezende and Mohamed 2015) or

structured approximations that are informed by the form of the model (Ambrogioni et al. 2021).

TreeFlow leverages the stochastic gradient optimizers already implemented in TensorFlow and the variational inference objectives in TensorFlow Probability. The discrete topology element of phylogenetic trees is an obstacle in the usage of these algorithms, which are typically restricted to continuous latent variables. Often, the phylogenetic tree topology is not the latent variable of interest, and is not a significant source of uncertainty (Yang et al. 2000). This can be the case when divergence times or other substitution or speciation model parameters are the focus. In this scenario, useful results can be obtained by performing inference with a fixed tree topology, such as one obtained from fast maximum likelihood methods. This is the approach taken by the NextStrain platform (Hadfield et al. 2018), which uses the scalability afforded by a fixed tree topology to allow large-scale rapid phylodynamic analysis of pathogen molecular sequence data. Using a variational approximation with a fixed topology in TreeFlow makes the application of ADVI to phylogenetic models straightforward. When using variational inference in TreeFlow, care must be taken if it is possible the tree topology for a given dataset is difficult to infer by assessing how sensitive the inference results are to it.

Although TreeFlow's current variational inference scheme is limited to a fixed tree topology, the rest of the library is not constrained in this manner. The Tensor-based tree representation means that the topology is part of TensorFlow's computational graph. Additionally, computations such as the phylogenetic likelihood are implemented with support for a dynamic tree topology. This means that existing approaches to variational inference over tree topologies (Zhang and Matsen IV 2018b) could be implemented using TreeFlow's model specification and likelihood evaluation, extending the library to support topology inference in the future.

Model approximations

TreeFlow implements posterior approximations for ADVI using TensorFlow Probability's bijector framework to transform a base distribution. Approximations are constructed automatically from probabilistic models in the form of TensorFlow Probability joint distribution objects. A base distribution, by default an uncorrelated Normal, is split into the variables of the joint distribution, and transformed to match the support of the model variables. The base distribution for the phylogenetic tree is transformed using a chain of bijectors. Learnable parameters to optimize are introduced by adding an initial trainable transformation to the base distribution. For the mean field approximation, this is an affine elementwise shifting and scaling operation.

The mean field approximation treats every coordinate of the base distribution as independent, and so cannot represent posterior correlation at all. TreeFlow therefore also provides a *full rank* approximation, in which the trainable transformation is a general affine map with a lower-triangular scale matrix, allowing the approximation to represent linear correlation between every pair of coordinates. For a fixed-topology phylogenetic model this is an awkward fit to the problem. The number of variational parameters is quadratic in the number of taxa and is dominated by the covariances between node heights, whereas the correlation structure that matters most for parameter estimation is concentrated in a few coordinates: the sequence data constrain the product of the evolutionary rate and the age of the tree far better than either quantity separately, so the clock rate, the tree prior's rate parameter and the root height are strongly correlated in the posterior.

TreeFlow's third approximation, which we call *root full rank*, is built around this observation. It places a full covariance block over the model parameters together with the root height of the tree, and treats the remaining node-height ratios as independent. The number of variational parameters is then linear in the number of taxa, as for the mean field approximation, while the correlations between the overall scale of the tree and the model parameters are represented as they are under the full rank approximation. Models

may additionally contain parameters that are replicated once per branch — an uncorrelated relaxed clock, or the per-lineage transition-transversion ratio model of the carnivores analysis below — whose dimension grows with the tree in the same way the node heights do. These are excluded from the covariance block by naming them when the approximation is constructed, and are approximated independently alongside the node-height ratios. The three approximations are compared on both of the datasets analysed below in the Supplementary Appendix (Figures S3 and S4).

ADVI opens the door to using TensorFlow's neural network framework to implement deep-learning-based variants such as variational inference with *normalizing flows* (Rezende and Mohamed 2015), which transform the base distribution through invertible trainable neural network layers to better approximate complex posterior distributions. Approximations based on *inverse autoregressive flows* (Kingma et al. 2016) are implemented in TreeFlow, but since their computational and memory requirements scale quadratically with the number of model variables they are not ideal for large phylogenetic analyses.

Biological examples

We used TreeFlow to perform fixed-topology phylogenetic analyses of two datasets. The first is an alignment of 62 mitochondrial DNA sequences from carnivores (Suchard and Rambaut 2009). The second is a dataset of 980 influenza sequences (Vaughan et al. 2014). In both datasets maximum likelihood unrooted topologies are estimated using RAxML (Stamatakis 2014). These topologies are rooted with Least Squares Dating (To et al. 2016).

Probabilistic modelling and model selection

In the carnivores dataset, we demonstrate the flexibility of probabilistic modelling with TreeFlow by investigating variation in the ratio of transitions to transversions in the nucleotide substitution process across lineages in the tree. Early maximum likelihood

analyses of mitochondrial DNA in primates detected variation in this ratio, but without a biological basis, it was attributed to a saturation effect (Yang and Yoder 1999). A later simulation-based investigation showed that this was a reasonable explanation, which exposed the limitations of nucleotide substitution models for estimating the length of branches deep in the tree (Duchêne et al. 2015).

This problem could be approached by means of Bayesian model comparison; a lack of preference for a model allowing between-lineage variation of the ratio could indicate that the substitution model lacks the power to separate variation 'signal' from saturation 'noise'. Firstly, we construct a standard phylogenetic model with a HKY substitution model with a single transition-transversion ratio parameter (*kappa*). We perform inference using ADVI, and also using MCMC as implemented in BEAST 2 (v2.7.7; (Bouckaert et al. 2019)). Since this model is of the form of a standard phylogenetic analysis, it could be fit using TreeFlow's command line interface. Figure 2 compares the marginal parameter estimates obtained from TreeFlow and BEAST 2. The variational and MCMC estimates agree closely for every parameter of the substitution and site rate models, including all four base frequencies, and the two estimates of the parameter of interest, kappa, are almost indistinguishable. The one variable whose distribution differs appreciably is the tree height, for which the variational estimate is more concentrated than the MCMC estimate and does not reproduce its upper tail. The MCMC posterior for the tree height is noticeably right-skewed, and no affine transformation of a Normal base distribution through the node height ratio transformation can reproduce an arbitrary skew, so some discrepancy of this kind is expected from any of the approximation families TreeFlow provides; the corresponding comparison for the mean field and full rank families is given in Supplementary Appendix Figure S3.

We then altered this model to estimate a separate ratio for every branch of the tree, that is, one independent kappa parameter per branch; the implementation of this was simple and scalable as a result of TensorFlow's vectorization and broadcasting

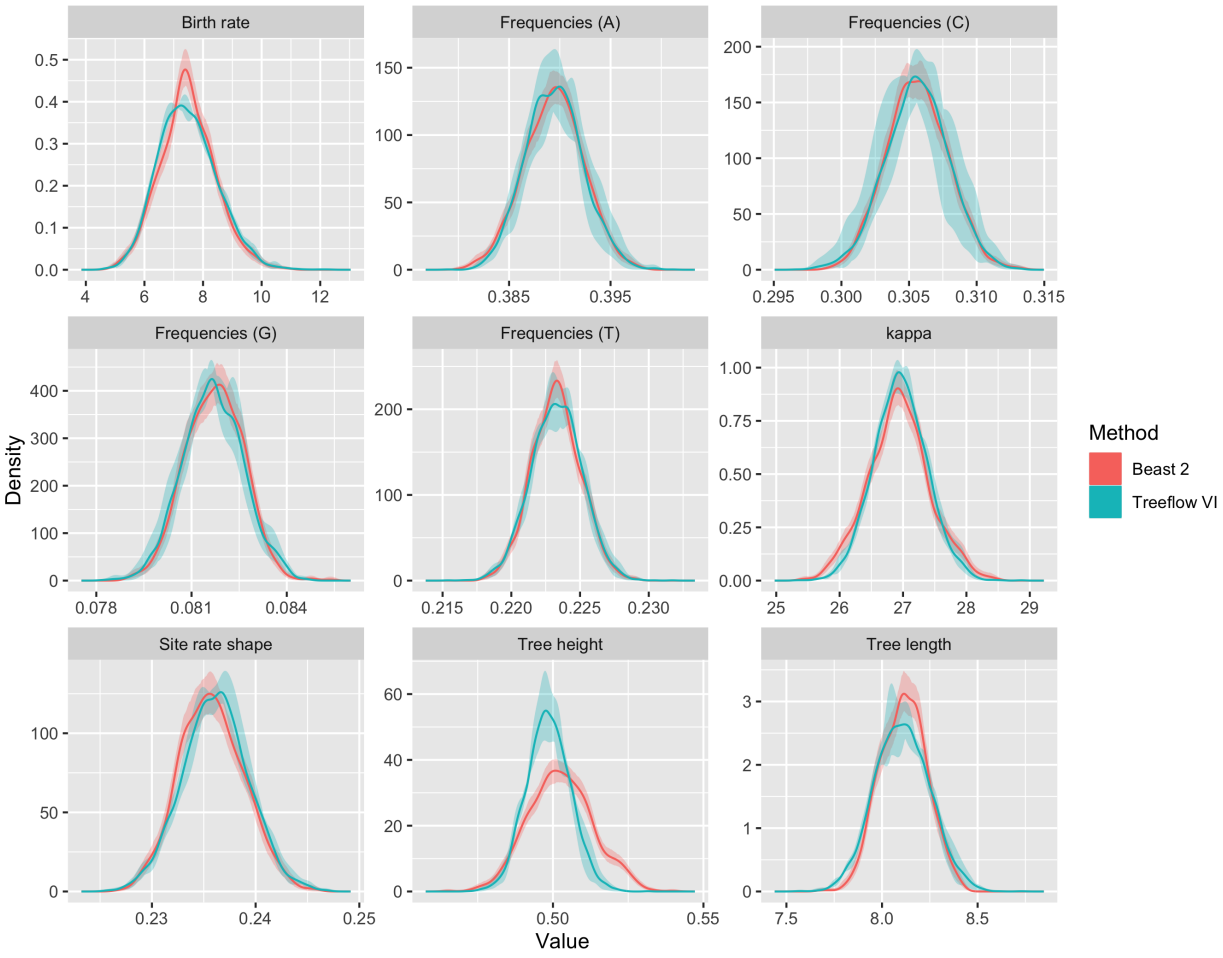
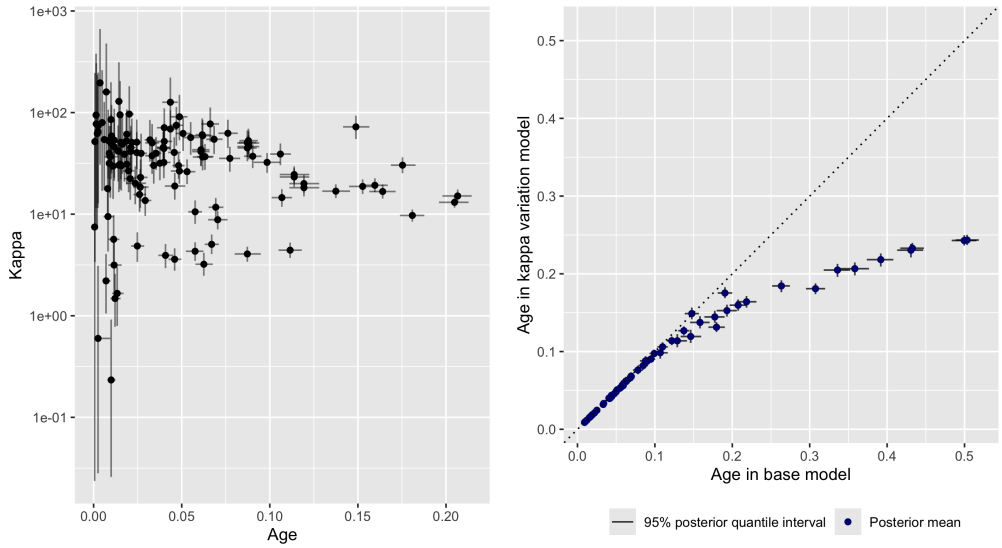


Fig. 2: Posterior marginal parameter estimates for the carnivores base model (a single transition/transversion ratio, kappa, shared across the whole tree), comparing TreeFlow variational inference (Treeflow VI, teal) with BEAST 2 MCMC (Beast 2, red). The solid curves show the estimated marginal posteriors and the shaded bands represent the Monte Carlo error of each estimate. For BEAST 2 the band is obtained by bootstrapping the MCMC samples; for TreeFlow VI it is estimated from the variability between four independent variational runs, with the curve showing the posterior pooled across those runs. The variational runs use the root full rank approximation; the corresponding comparison of all three approximation families is given in Supplementary Appendix Figure S3.

functionality. The model definition code for the base model and the small changes required to model variation in the kappa parameter across lineages are shown in Supplementary Appendix Figure S2. We compared the models using estimates of the marginal likelihood (MacKay 2003). The *marginal likelihood*, or *evidence*, the integral of the likelihood of the data over model parameters, is typically analytically intractable and challenging to compute using MCMC methods (Xie et al. 2011). The closed-form approximation to the posterior distribution provided by variational inference can be used as a proposal distribution for importance sampling, a utility provided by TreeFlow. Concretely, one draws samples from the fitted approximation and weights each sample by the ratio of the model's joint density (the likelihood multiplied by the prior) to the density of the approximation; the average of these importance weights is an unbiased Monte Carlo estimate of the marginal likelihood, obtained directly rather than as the lower bound that is optimized during variational inference (Burda et al. 2015). Because the importance weights account for the discrepancy between the variational approximation and the true posterior, this produces more accurate marginal likelihood estimates than the variational bound alone.

The estimated per-lineage kappa parameters are shown in Figure 3a. Kappa estimates decline with increasing age of the lineage, which agrees with the results of the previous studies. The estimated log marginal likelihood for the base model was -193606.6 and for the model with kappa variation it was -192414.7, indicating that the model with transition-transversion ratio variation across lineages is preferred. This means that, under the other components of this model, the data supports variation in the ratio. This does not necessarily imply that transition-transversion ratio in the true generative process of the data varies in the same way, but could be a useful consideration in designing more sophisticated models of nucleotide substitution. The uncertainty in the lineage age estimates grows for branches deeper in the tree (the horizontal error bars in Figure 3a), supporting previous conclusions that nucleotide substitution models are unable to



(a) Estimated per-branch kappa against the age of each branch's lineage. Each point is the posterior median of the kappa parameter for one branch; the vertical bars show its 95% credible interval and the horizontal bars show the 95% credible interval of the lineage age. Age is the height of the branch above the present, measured in expected substitutions per site (the tree is estimated in units of substitutions and is not calibrated to time); it is therefore not the branch length.

(b) Internal node age estimates under the base (single-kappa) model against those under the per-branch kappa variation model. Each point is the posterior mean age of one internal node, with bars showing the 95% posterior quantile interval; the dotted line is the identity. Points falling below the identity for older nodes show that the kappa variation model infers younger ages for deeper nodes.

Fig. 3: Parameter estimates for the carnivores per-branch transition/transversion ratio (kappa) variation model. In this model every branch is assigned its own independent kappa parameter, drawn from a shared log-normal prior sampled once per branch; kappa is not shared between branches and does not follow any model of change over time or at speciation events. Panel (a) shows how the per-branch kappa estimates vary with the age of the lineage, and panel (b) shows the effect of allowing per-branch kappa variation on the estimated internal node ages.

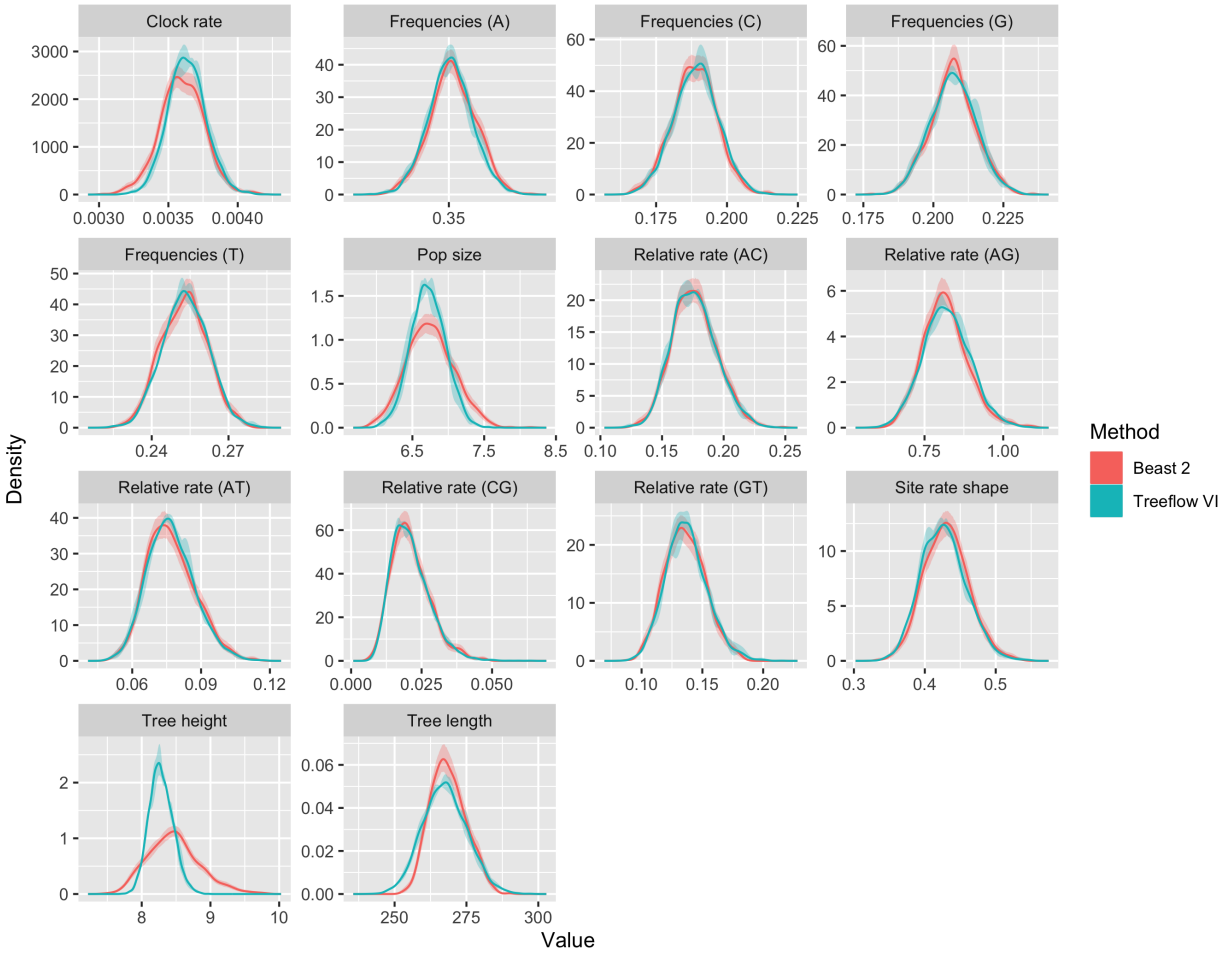
effectively estimate the number of substitutions on older branches (Duchêne et al. 2015). The uncertainty in the per-branch kappa estimates shows the opposite pattern: it is largest for the youngest, shortest branches, which accumulate too few substitutions to constrain kappa, and is smaller on older branches even as the kappa point estimates themselves decline. Figure 3b shows that the kappa variation model shortens older branches, leading to a substantially reduced overall tree height estimate, with proportionally similar uncertainty in node height estimates. This is not a proper dating analysis because it does

not account for uncertainty in the mutation rate and lacks time calibration. However, it is evident that the inclusion or exclusion of heterogeneity in the transition-transversion ratio affects the time estimates. This example demonstrates the power of TreeFlow's probabilistic modelling approach: extending the standard model to include per-lineage kappa variation required only a few lines of code changes (Supplementary Appendix Figure S2), and the variational inference and marginal likelihood estimation machinery could be applied to the new model without any additional implementation effort.

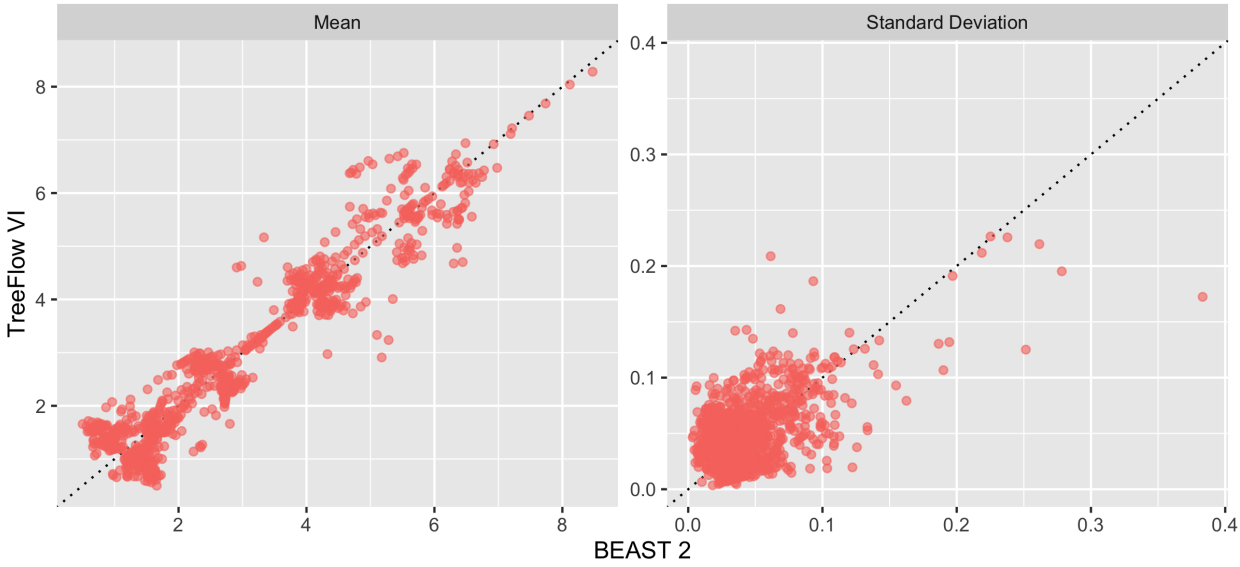
Scalable inference

In the analysis of the 980-taxon influenza dataset, we demonstrate the scalability of variational inference. We performed variational inference using TreeFlow's command line interface. The model used for inference included a coalescent prior with a constant population size on the tree, a strict molecular clock, a discretized Gamma model of site rate variation and a GTR substitution model. The YAML model definition code for this model, including parameter priors, is shown in Supplementary Appendix Figure S1. This parameterization of the GTR substitution model (here named `gtr_rel`), with independent priors on five of the relative rates and one held fixed to 1, is used to allow comparison of parameter estimates with BEAST 2. It is also possible to use a six-rate GTR parameterisation with a Dirichlet prior in TreeFlow such as used in other phylogenetics software (Huelsenbeck and Ronquist 2001; Höhna et al. 2016).

Figure 4a shows the marginal parameter estimates obtained from this dataset, compared to those obtained from a fixed-topology MCMC analysis using BEAST 2. The posterior distributions of the substitution and site rate model parameters are approximated with high fidelity, as are all four base frequencies. The stationary frequency of T is worth remarking on specifically: under a mean field approximation its uncertainty is noticeably underestimated in both this analysis and the carnivores analysis, whereas the other three base frequencies are approximated well (Supplementary Appendix Figures S3



(a) Marginal posterior distributions for the model parameters, comparing TreeFlow variational inference (Treeflow VI, teal) with BEAST 2 MCMC (Beast 2, red). As in Figure 2, the shaded band around each curve represents the Monte Carlo error of that estimate: for BEAST 2 it is obtained by bootstrapping the MCMC samples, and for TreeFlow VI from the variability between four independent variational runs, with the curve showing the posterior pooled across those runs. The variational runs use the root full rank approximation; Supplementary Appendix Figure S4 gives the corresponding comparison of all three approximation families.



(b) Per-node summaries of the internal node height posteriors, plotting the TreeFlow VI estimate against the BEAST 2 estimate for each node. The left panel compares the posterior means and the right panel the posterior standard deviations; the dotted line in each panel is the identity. Points above the identity in the right panel are node heights

and S4). The base frequencies are constrained to sum to one and are represented in the variational approximation through a transformation in which the final coordinate is determined by the others, a dependence the mean field family cannot represent; the root full rank approximation places these coordinates in its covariance block, and the discrepancy disappears.

The uncertainty in the tree height and the coalescent effective population size remains underestimated. That this survives an approximation which represents the correlation between these parameters explicitly indicates that the residual error is not a matter of missing linear correlation. Two other mechanisms are at work. The MCMC posteriors for both quantities are right-skewed, and no affine transformation of a Normal base distribution can reproduce an arbitrary skew. In addition, the objective function used in ADVI, the KL divergence between the approximating and posterior distributions, heavily penalises regions of the approximation where there is no posterior support, which typically results in low uncertainty estimates.

Figure 4b compares the divergence time estimates. Under the root full rank approximation the root height is included in the covariance block, but the remaining node heights are represented by independent ratios transformed through the node height ratio transformation to respect the time constraints of the tree. The true posterior is almost certainly not of this form, so the approximation introduces error.

The posterior means of the divergence times are estimated reliably. Across the 979 internal nodes the variational and MCMC posterior means have a Pearson correlation of 0.95 and a regression slope of 0.95, with a median absolute relative difference of 13%. The per-node uncertainties are estimated less reliably, with a correlation of 0.56 between the two sets of standard deviations, and the direction of the error depends on how well determined the node is. For the quarter of nodes whose heights BEAST 2 estimates most precisely, the variational standard deviation is a median of 2.5 times the MCMC value, so the approximation is over-dispersed; for the quarter estimated least precisely it is

0.9 times, slightly under-dispersed. The error is therefore not a uniform underestimate of uncertainty, as it appears under a mean field approximation, but a failure to track how uncertainty varies from node to node — which is what would be expected when the node height ratios are treated as independent. Divergence time point estimates from TreeFlow's variational inference can be used with reasonable confidence; per-node uncertainties should be treated more cautiously. Variational approximations that represent dependence among the node heights themselves, rather than only between the root height and the model parameters, would improve the quality of these estimates.

BEAST 2 MCMC sampling was run for 30,000,000 iterations using the standard single-threaded MCMC routine. Convergence was checked using *effective sample size* (ESS), which was computed for all parameters using ArviZ (Kumar et al. 2019). BEAST 2's built-in operator autotuning was used, and multiple preliminary runs were performed to manually adjust operator weights to improve ESSs. Generally, the minimum acceptable ESS is 100, though 200 is preferred (Drummond and Bouckaert 2015). The analysis resulted in a minimum effective sample size of 103. The wall clock runtime for the BEAST 2 analysis was 3 hours and 58 minutes. Convergence for the TreeFlow-based variational inference analysis was assessed by monitoring the evidence lower bound (ELBO) over the optimization routine and visually examining parameter traces. In practice, TreeFlow's command line interface reports an ELBO estimate at the end of optimization, and the full trace of the loss and the variational parameters over the optimization can be saved (using the `-trace-output` option) and plotted; convergence is diagnosed by checking that the ELBO has reached a stable plateau and that the variational parameters have stopped drifting, and, if in doubt, by rerunning with a larger number of optimization steps. Step-by-step guidance on monitoring the convergence of variational inference in TreeFlow, including interpreting these traces, is provided in the online manual. The TreeFlow analysis converged in 22,000 iterations, which took 21 minutes and 36 seconds; running the full 60,000 iterations took 58 minutes and 56 seconds. Reaching convergence was therefore

around 11 times faster than the BEAST 2 analysis was to reach the effective sample sizes reported above, and even the full optimization run, continued well past the point at which the ELBO had plateaued, finished in a fraction of the MCMC runtime. The comparison in fact understates the difference, for two reasons. The BEAST 2 time is for the single production run only, and excludes the multiple preliminary runs that were needed to tune operator weights and reach acceptable effective sample sizes, whereas the variational analysis was run once and required no such manual tuning. It also stops at a minimum effective sample size below the value of 200 usually preferred, so a production MCMC analysis of this dataset would need to run longer still. This demonstrates that variational inference with TreeFlow is an efficient option for large phylogenetic datasets, with the further advantage that its optimization-based approach is expected to scale more favourably than random-walk MCMC as datasets grow. TreeFlow's phylogenetic likelihood implementation performs well enough to be useful in real-world inference scenarios, and the entire analysis was specified using TreeFlow's YAML model definition format (Supplementary Appendix Figure S1) without any custom code.

Benchmarks

The phylogenetic likelihood is the computation that dominates the computational cost of model-based phylogenetic inference. We benchmark the performance of our TensorFlow-based likelihood implementation against specialized libraries. Since gradient computations are of equal importance to likelihood evaluations in modern automatic inference regimes, we also benchmark computation of derivatives of the phylogenetic likelihood, with respect to both the continuous elements of the tree and the parameters of the substitution model. A clear difference emerges in the implementation of derivatives; TreeFlow's are based on automatic differentiation while bespoke libraries need analytically-derived gradients. Therefore, we do not necessarily expect our implementation to be as fast as bespoke software, but it does not rely on analytical derivation of gradient

expressions for every model and therefore automatically supports a wider range of models.

We benchmark both of the phylogenetic likelihood implementations described in the Tree traversals section: the portable, pure-TensorFlow TensorArray traversal, which we refer to as TreeFlow, and the compiled native C++ operation, which we refer to as TreeFlow (native).

We compare the performance of TreeFlow's likelihood implementation against BEAGLE (Ayres et al. 2019). BEAGLE is a library for high-performance computation of phylogenetic likelihoods. Recent versions of BEAGLE implement analytical gradients with respect to tree branch lengths (Ji et al. 2020). We use BEAGLE via the `bito` software package (Matsen et al. 2023), which provides a Python interface to BEAGLE's branch length gradients and also numerically computes derivatives with respect to substitution model parameters using a finite differences approximation. This hybrid approach has been shown to perform as well as fully analytical substitution model parameter gradients on large datasets (Fourment et al. 2023).

We also compare TreeFlow with a simple likelihood implementation (Swanepoel 2023) based on another automatic differentiation framework, JAX (Bradbury et al. 2018). JAX uses an eager execution model where execution of an operation immediately results in the computation being performed. JAX can additionally trace and compile a function ahead of execution with the XLA compiler (just-in-time, or JIT, compilation); we benchmark both the eager and JIT-compiled variants, denoted JAX and JAX (JIT). In contrast, TensorFlow's function mode, which is used by TreeFlow in the benchmarks, constructs a graph representation of the computation which is processed and can then be executed. As a result the phylogenetic likelihood in JAX can be implemented with native Python control flow even with a dynamic tree topology; the JIT-compiled variant, however, specialises the compiled kernel to a single fixed topology, unrolling the traversal so that the kernel must be regenerated whenever the topology changes.

Benchmarks are performed on simulated data. Simulation allows the generation of a

large number of data replicates with appropriate properties. Since we want to investigate the scaling of likelihood and gradient calculations with respect to the number of sequences, we simulate data for a range of sequence counts. Sequence counts are selected as increasing powers of 2 to better display asymptotic properties of the implementations. We simulate data with sequence counts ranging from 32 to 2048. Trees are simulated under a constant-size coalescent model for a given number of taxa, and then nucleotide sequences of length 1000 are simulated under a strict molecular clock. Tree and sequence simulation are performed using TreeFlow itself: coalescent genealogies are drawn with a serial-sampling waiting-time simulator, and alignments are sampled from the same phylogenetic likelihood distribution that is used to score them, so the simulated data is by construction consistent with the models being evaluated and the benchmark has no external dependencies.

We benchmark four distinct computations on these datasets. Firstly, for each replicate we compute the phylogenetic likelihood under a very simple model of sequence evolution. This uses a Jukes-Cantor model of nucleotide substitution and no model of site rate variation. Secondly, we calculate derivatives with respect to branch lengths under this simple model. Thirdly, we compute the likelihood under a more sophisticated substitution model, with a discretized Weibull distribution with four rate categories to model site rate variation and a GTR model of nucleotide substitution. This Weibull distribution is used for site rate variation in `bito` instead of the widely used Gamma distribution because its inverse cumulative distribution function (required for computing rates for the discretized categories) has a simpler form. Because this is a minor part of the computation, only performed once for each rate category per likelihood evaluation, TreeFlow's scaling should be unaffected if the standard Gamma distribution is used. Finally, we compute derivatives with respect to both branch lengths and the parameters of the substitution model (the six GTR relative substitution rates, four base nucleotide frequencies, and the Weibull site rate shape parameter).

All reported times are the cost of a *single* evaluation of the computation in

question. Each computation is timed individually over 100 repeated calls on one fixed input, and we take the minimum of those timings as the estimate for that dataset. Repeating a single input is appropriate here because the cost of a tree traversal depends on the size and shape of the tree rather than on the branch length values themselves, so repetition isolates the per-call cost from scheduling jitter; taking the minimum further discards positive noise and excludes the one-off tracing or compilation cost incurred on the first call, so the times reported are steady-state costs. This procedure is repeated on 10 independently simulated datasets per taxon count, and the figures and tables below report the mean across those datasets.

Figure 5 shows the results of the benchmarks with a log scale on both axes, and Table 1 reports representative runtimes together with the fitted log–log slopes. Considering first the phylogenetic likelihood, `bito`/BEAGLE is the fastest implementation, as expected for a highly optimized special-purpose library written in native code. TreeFlow's native operation is within a small constant factor of it: around three to four times slower for likelihood evaluation on the larger trees, with the gap widening to roughly an order of magnitude on the smallest trees, where fixed per-call overhead in the TensorFlow dispatch rather than the traversal itself dominates the runtime. It is in turn faster than TreeFlow's portable TensorArray implementation by a factor that grows from a few times on the smallest trees to one to two orders of magnitude on trees of a few hundred taxa and above. The TensorArray implementation is the slowest of the TreeFlow variants but, as discussed below, still scales near-linearly.

The picture changes for the gradient of the more complex GTR/Weibull model, which is the computation most relevant to gradient-based inference: for variational inference and Hamiltonian Monte Carlo the gradient is the primary quantity computed at each iteration. For this computation TreeFlow's native operation is the fastest of all implementations, faster even than `bito`/BEAGLE (Table 1). The two are comparable on the smallest trees, but the native operation pulls ahead as the trees grow, reaching a factor

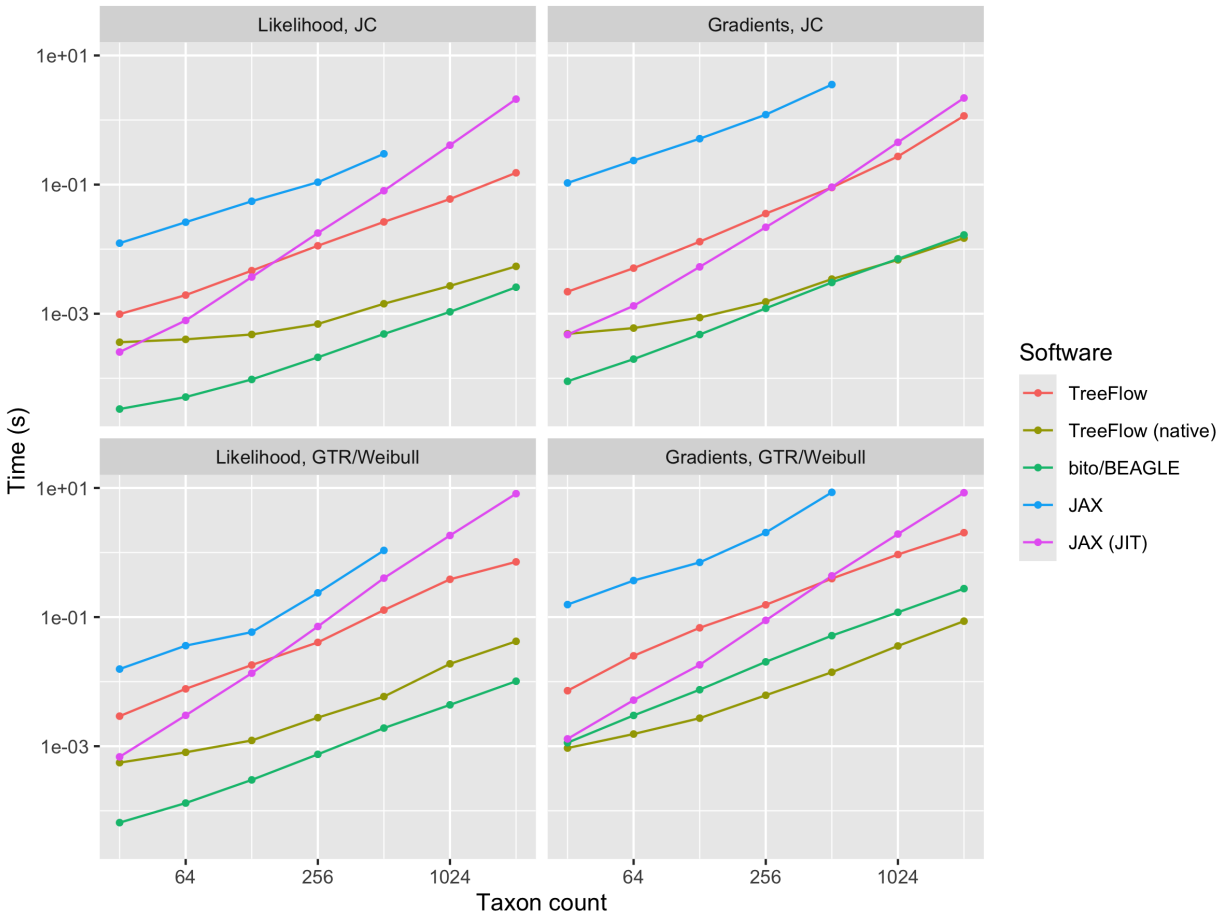


Fig. 5: Times from the phylogenetic likelihood benchmark, on log-log axes. Each point is the time for a single evaluation of the computation on an alignment of 1000 sites, averaged over 10 simulated datasets. Each panel corresponds to one of the four benchmarked computations (likelihood and gradient evaluation, under the simple JC model and the more complex GTR/Weibull model), with the five implementations plotted as separate series: TreeFlow's portable TensorArray traversal (TreeFlow) and its native C++ operation (TreeFlow (native)), `bito`/BEAGLE, and the eager (JAX) and JIT-compiled (JAX (JIT)) JAX implementations. This layout makes the differences in scaling (the slope of each series) between the implementations directly comparable within each computation. Eager JAX is only run up to 512 taxa as its runtime becomes prohibitive on larger trees.

Model	Computation	Method	Time per evaluation, s (512 taxa)	log-log slope
JC	Likelihood	TreeFlow	0.0264	1.22
		TreeFlow (native)	0.00143	0.67
		bito/BEAGLE	0.000485	1.07
		JAX	0.300	1.12
		JAX (JIT)	0.0804	2.20
	Gradients	TreeFlow	0.0902	1.48
		TreeFlow (native)	0.00344	0.85
		bito/BEAGLE	0.00305	1.27
		JAX	3.54	1.25
		JAX (JIT)	0.0913	2.05
GTR/Weibull	Likelihood	TreeFlow	0.129	1.36
		TreeFlow (native)	0.00587	1.07
		bito/BEAGLE	0.00191	1.24
		JAX	1.08	1.49
		JAX (JIT)	0.401	2.29
	Gradients	TreeFlow	**0.394**	**1.34**
		TreeFlow (native)	**0.0140**	**1.11**
		bito/BEAGLE	**0.0516**	**1.33**
		JAX	**8.56**	**1.40**
		JAX (JIT)	**0.435**	**2.13**

Table 1: Results of phylogenetic likelihood benchmark. Times are for a single evaluation of the computation on a 512-taxon tree with 1000 sites, in seconds, quoted to three significant figures and averaged over 10 simulated datasets; slopes are fitted to log-transformed times against log-transformed taxon counts across the full range of taxon counts. Times for gradients for the GTR/Weibull are highlighted as they are the most relevant computation for gradient-based inference on real data.

of around three to four from a few hundred taxa upwards. The reason is the difference in how substitution-model parameter derivatives are obtained: `bito` computes them by finite differences (Fourment et al. 2023), performing additional likelihood evaluations per parameter, whereas TreeFlow obtains the entire gradient, with respect to branch lengths and all substitution-model parameters, in a single reverse-mode automatic-differentiation pass at close to the cost of one likelihood evaluation. This advantage grows with the number of parameters, so TreeFlow is an especially attractive choice for richly parameterized models (e.g. amino acid substitution models (Adachi and Hasegawa 1996))

or models where the number of parameters grows with the number of sequences, as in the carnivore example above. Even TreeFlow's portable TensorArray implementation, though slower in absolute terms, benefits from the same automatic-differentiation advantage on the many-parameter gradient.

The JAX-based implementations are the slowest overall. In eager mode JAX is several times to nearly two orders of magnitude slower than TreeFlow's TensorArray implementation, with the largest gaps on the gradient computations, and its runtime becomes prohibitive on large trees, so we cap the eager-JAX benchmark at 512 taxa. JIT-compiling the JAX computation with XLA yields a large speed-up at small tree sizes, so that at a few hundred taxa it is comparable to TreeFlow's TensorArray implementation, but its runtime scales super-linearly, with a log–log slope close to two (Table 1), so that on the largest trees it becomes the slowest implementation of all. This behaviour illustrates the quadratic-complexity pitfall described in the Tree traversals section: a naive automatic-differentiation implementation of a tree traversal incurs a cost that grows with the square of the number of taxa. Just-in-time compilation reduces the constant factor of the computation, which is what gives the JIT-compiled variant its advantage at small taxon counts, but it does not change the underlying algorithmic complexity, so the quadratic scaling eventually dominates and the small-taxa advantage is lost. TreeFlow avoids this pitfall by expressing the traversal with TensorFlow control-flow primitives, a `while_loop` in the portable implementation and a single fused operation in the native one, which keeps the memory and computational cost, and hence the scaling, linear in the number of taxa. These results indicate that graph- and loop-based execution is better suited to tree-structured computations than per-topology just-in-time compilation.

Table 1 shows the coefficients obtained from fitting a linear model to the benchmark times where the predictor is the log-transformed number of sequences and the target is the log-transformed runtime. The slope parameter estimates from these fits are a rough empirical estimate of the polynomial degree of the computational scaling. The slopes

for all of the TreeFlow and `bito`/BEAGLE computations are well below 2, indicating roughly linear scaling with the size of the data; the native operation has the smallest slopes of any implementation. By contrast the JIT-compiled JAX implementation has slopes close to 2 across all four computations, consistent with the super-linear behaviour noted above and with the topology-specialised graph it compiles. For the JC benchmark, the only gradients computed are with respect to branch lengths, so the whole computation for `bito`/BEAGLE is the analytical linear-time branch gradient. In this case TreeFlow's TensorArray slope is slightly steeper than that of `bito`/BEAGLE (Table 1). Both slopes are close to one and far below the value of two that a naive quadratic implementation would produce, so we regard the difference as small and within the noise of the log-log fits; the TensorArray-based implementation therefore still achieves near-linear-time branch gradients through automatic differentiation, matching the scaling of the hand-derived analytical gradient even if it is a constant factor slower.

Taken together, these results show that TreeFlow's native operation makes automatic-differentiation-based phylogenetics competitive with a specialized native library, and faster than it for the most inference-relevant computation, the gradient of a many-parameter model, while the portable TensorArray implementation provides a dependency-free alternative with the same near-linear scaling for use where a compiled operation is unavailable or a dynamic topology is required.

Discussion

The probabilistic modelling approach enabled by the TreeFlow Python API in conjunction with TensorFlow Probability has similar goals to other phylogenetic probabilistic modelling software. Two notable examples, RevBayes (Höhna et al. 2016) and Blang (Bouchard-Côté et al. 2022) provide a generic model definition and inference engine which also support phylogenetic models and tree inference. Some notable differences exist between TreeFlow and these which may prove advantageous for certain users. Firstly,

RevBayes requires the user to manually specify an MCMC operator scheme, including parameters and weights. This may require substantial user time and expertise to achieve correct and efficient inference. Blang and TreeFlow, on the other hand, are focused on automatic inference methods that require less user input. Secondly, TreeFlow and its Python model specification API are implemented in a widely used computational framework (TensorFlow) and programming language (Python). As a result it can potentially leverage other computational tools implemented in TensorFlow, including less common distributions, machine learning models, differential equation solvers and alternative inference schemes. The automatic differentiation capabilities TensorFlow affords mean TreeFlow can interface with the large and growing collection of models that rely on gradient-based inference. On the other hand, the choice of computational framework incurs a performance cost as evidenced by the results of benchmarking. Blang models can make use of external Java code, but the ecosystem of related software in Java is less developed, and RevBayes does not provide any functionality for interfacing with software that is not implemented in the Rev language.

The combination of flexibility and scalability of libraries like TreeFlow has the potential to be useful for rapid analysis of large modern genetic datasets, such as those assembled for tracking infectious diseases (Hadfield et al. 2018). For this purpose, TreeFlow's true power would be unlocked with the implementation of improved modelling and inference tools for evolutionary rates and population dynamics.

For principled dating analyses, uncertainty in evolutionary rate parameters must be considered. Posterior distributions involving these parameters have significant correlation structure which is ignored by variational inference using a mean-field posterior approximation, and these parameters typically receive special treatment in MCMC sampling routines (Drummond et al. 2006; Zhang and Drummond 2020). The root full rank approximation introduced here is a structured variational family aimed at exactly this problem: it captures the correlation between the evolutionary rate and the age of the

tree using a covariance block over the model parameters and the root height, at a cost that remains linear in the number of taxa. The remaining approximation error is concentrated in structure this family does not represent.

The clearest limitation concerns models whose parameters are replicated per branch, and whose dimension therefore grows with the tree. An uncorrelated relaxed clock, or the per-lineage transition-transversion ratio model analysed above, introduces exactly the kind of rate-time correlation that motivates the covariance block, but at a dimension that would restore the quadratic cost the block was designed to avoid; TreeFlow consequently approximates these parameters independently. The correlation between a branch's rate and the ages of the nodes it separates is therefore not captured, which is a substantive limitation for relaxed-clock dating analyses in particular. Extending the structured family to represent this dependency — for example with a covariance structure that follows the tree, so that each branch parameter is correlated with the node heights adjacent to it rather than with all of them — is a natural next step. Correlations that are not linear in the unconstrained coordinates are also outside the reach of any affine family, and could be addressed by applying normalizing flows (Rezende and Mohamed 2015) to model more complex dependencies.

Additionally, implementing flexible models for population dynamics, such as nonparametric coalescent models (Drummond et al. 2005; Minin et al. 2008; Gill et al. 2013), could provide valuable insights from large datasets. These models would result in complex posterior distributions, which would need to be accounted for in a variational inference scheme. If these coalescent models could be made to work effectively in TreeFlow, the probabilistic graphical modelling paradigm would allow for rapid computational experimentation with novel models of time-varying population parameters.

Finally, the most significant functionality for phylogenetic inference missing from TreeFlow is inference of the tree topology. TreeFlow's tree representation could allow for the topology to become an estimated parameter, such as in existing work on variational

Bayesian phylogenetic inference (Zhang and Matsen IV 2018b). Efficiently implementing the computations required for these algorithms in the TensorFlow framework would be a major computational challenge, but would be supported by TreeFlow's representation of the tree topology as a dynamic variable inside the computational graph. A promising direction would be to combine TreeFlow's existing model components and gradient computations with a variational distribution over tree topologies, such as the subsplit Bayesian network approach (Zhang and Matsen IV 2018a), enabling joint inference of topology and continuous parameters.

Conclusion

TreeFlow is a software library that provides tools for statistical modelling and inference on phylogenetic problems. Probabilistic graphical modelling provides flexible model definition involving phylogenetic trees and molecular sequences, and automatic differentiation enables the application of modern scalable inference algorithms with a fixed tree topology. TreeFlow's automatic differentiation-based implementations of core gradient computations provide reasonable performance compared to specialized phylogenetic libraries. These building blocks have the potential to be used in valuable phylogenetic analyses of large and complex genetic datasets, and with the next generation of inference methods that can accelerate the inference of tree topologies using gradient information.

Availability

TreeFlow is an open-source Python package. Instructions for installing TreeFlow, and examples of using it as a library and command line application can be found at `https://github.com/christiaanjs/treeflow`. The manual for TreeFlow can be found at `https://treeflow.readthedocs.io`.

Data availability

Datasets, model definitions, starting values, and BEAST 2 XML files used in the biological examples are available at `https://github.com/christiaanjs/treeflow-paper/releases/tag/v0.0.1` or `https://doi.org/10.5061/dryad.v6wwpzh0p`.

Conflicts of Interest

The authors declare that there are no conflicts of interest.

References

Abadi M, Barham P, Chen J, Chen Z, Davis A, Dean J, Devin M, Ghemawat S, Irving G, Isard M, Kudlur M, Levenberg J, Monga R, Moore S, Murray DG, Steiner B, Tucker P, Vasudevan V, Warden P, Wicke M, Yu Y, Zheng X. 2016. Tensorflow: a system for large-scale machine learning. In: Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation. OSDI'16. USA: USENIX Association. p. 265–283.

Adachi J, Hasegawa M. 1996. Model of amino acid substitution in proteins encoded by mitochondrial DNA. Journal of Molecular Evolution 42:459–468.

Ambrogioni L, Lin K, Fertig E, Vikram S, Hinne M, Moore D, van Gerven M. 2021. Automatic structured variational inference. In: International Conference on Artificial Intelligence and Statistics. PMLR. p. 676–684.

Ayres DL, Cummings MP, Baele G, Darling AE, Lewis PO, Swofford DL, Huelsenbeck JP, Lemey P, Rambaut A, Suchard MA. 2019. BEAGLE 3: Improved performance, scaling, and usability for a high-performance computing library for statistical phylogenetics. Systematic Biology 68:1052–1061.

Bergstra J, Breuleux O, Bastien F, Lamblin P, Pascanu R, Desjardins G, Turian J, Warde-Farley D, Bengio Y. 2010. Theano: a CPU and GPU math expression compiler. In: Proceedings of the 9th Python in Science Conference. p. 1–7.

Bingham E, Chen JP, Jankowiak M, Obermeyer F, Pradhan N, Karaletsos T, Singh R, Szerlip P, Horsfall P, Goodman ND. 2019. Pyro: Deep universal probabilistic programming. The Journal of Machine Learning Research 20:973–978.

Blei DM, Kucukelbir A, McAuliffe JD. 2017. Variational inference: A review for statisticians. Journal of the American Statistical Association 112:859–877.

Bottou L. 2010. Large-scale machine learning with stochastic gradient descent. In: Proceedings of COMPSTAT'2010. Springer. p. 177–186.

Bouchard-Côté A, Chern K, Cubranic D, Hosseini S, Hume J, Lepur M, Ouyang Z, Sgarbi G. 2022. Blang: Bayesian declarative modeling of general data structures and inference via algorithms based on distribution continua. Journal of Statistical Software 103:1–98.

Bouckaert R, Vaughan TG, Barido-Sottani J, Duchêne S, Fourment M, Gavryushkina A, Heled J, Jones G, Kühnert D, De Maio N, Matschiner M, Mendes FK, Müller NF, Ogilvie HA, du Plessis L, Popinga A, Rambaut A, Rasmussen D, Siveroni I, Suchard MA, Wu CH, Xie D, Zhang C, Stadler T, Drummond AJ. 2019. Beast 2.5: An advanced software platform for bayesian evolutionary analysis. PLOS Computational Biology 15:1–28. doi:10.1371/journal.pcbi.1006650.

Bradbury J, Frostig R, Hawkins P, Johnson MJ, Leary C, Maclaurin D, Necula G, Paszke A, VanderPlas J, Wanderman-Milne S, Zhang Q. 2018. JAX: composable transformations of Python+NumPy programs.

Burda Y, Grosse R, Salakhutdinov R. 2015. Importance weighted autoencoders. arXiv preprint arXiv:1509.00519 .

Carpenter B, Gelman A, Hoffman MD, Lee D, Goodrich B, Betancourt M, Brubaker M, Guo J, Li P, Riddell A. 2017. Stan: A probabilistic programming language. Journal of Statistical Software 76.

Dillon J, Langmore I, Tran D, Brevdo E, Vasudevan S, Moore D, Patton B, Alemi A, Hoffman M, Saurous RA. 2017. Tensorflow distributions. In: Workshop on Probabilistic Programming Languages, Semantics, and Systems (PPS 2018).

Dinh V, Bilge A, Zhang C, Matsen IV FA. 2017. Probabilistic path Hamiltonian Monte Carlo. In: International Conference on Machine Learning. PMLR. p. 1009–1018.

Drummond AJ, Bouckaert RR. 2015. Bayesian evolutionary analysis with BEAST. Cambridge University Press.

Drummond AJ, Chen K, Mendes FK, Xie D. 2023. LinguaPhylo: a probabilistic model specification language for reproducible phylogenetic analyses. PLOS Computational Biology 19:e1011226.

Drummond AJ, Ho SY, Phillips MJ, Rambaut A. 2006. Relaxed phylogenetics and dating with confidence. PLoS Biology 4:e88.

Drummond AJ, Rambaut A, Shapiro B, Pybus OG. 2005. Bayesian coalescent inference of past population dynamics from molecular sequences. Molecular Biology and Evolution 22:1185–1192.

Drummond AJ, Suchard MA, Xie D, Rambaut A. 2012. Bayesian phylogenetics with BEAUti and the BEAST 1.7. Molecular Biology and Evolution 29:1969–1973.

Duane S, Kennedy AD, Pendleton BJ, Roweth D. 1987. Hybrid Monte Carlo. Physics Letters B 195:216–222.

Duchêne S, Ho SY, Holmes EC. 2015. Declining transition/transversion ratios through time reveal limitations to the accuracy of nucleotide substitution models. BMC Evolutionary Biology 15:1–10.

Felsenstein J. 1981. Evolutionary trees from DNA sequences: a maximum likelihood approach. Journal of Molecular Evolution 17:368–376.

Fisher AA, Ji X, Nishimura A, Baele G, Lemey P, Suchard MA. 2023. Shrinkage-based random local clocks with scalable inference. Molecular Biology and Evolution 40:msad242.

Fourment M, Darling AE. 2019. Evaluating probabilistic programming and fast variational Bayesian inference in phylogenetics. PeerJ 7:e8272.

Fourment M, Swanepoel CJ, Galloway JG, Ji X, Gangavarapu K, Suchard MA, Matsen Iv FA. 2023. Automatic differentiation is no panacea for phylogenetic gradient computation. Genome biology and evolution 15:evad099.

Gill MS, Lemey P, Faria NR, Rambaut A, Shapiro B, Suchard MA. 2013. Improving bayesian population dynamics inference: a coalescent-based model for multiple loci. Molecular Biology and Evolution 30:713–724.

Goodman ND, Mansinghka VK, Roy D, Bonawitz K, Tenenbaum JB. 2008. Church: a language for generative models. In: Proceedings of the Twenty-Fourth Conference on Uncertainty in Artificial Intelligence. UAI'08. Arlington, Virginia, USA: AUAI Press. p. 220–229.

Hadfield J, Megill C, Bell SM, Huddleston J, Potter B, Callender C, Sagulenko P, Bedford T, Neher RA. 2018. Nextstrain: real-time tracking of pathogen evolution. Bioinformatics 34:4121–4123.

Hasegawa M, Kishino H, Yano Ta. 1985. Dating of the human-ape splitting by a molecular clock of mitochondrial DNA. Journal of Molecular Evolution 22:160–174.

Hastings WK. 1970. Monte Carlo sampling methods using Markov chains and their applications. Biometrika 57:97–109.

Höhna S, Drummond AJ. 2012. Guided tree topology proposals for bayesian phylogenetic inference. Systematic Biology 61:1–11.

Höhna S, Heath TA, Boussau B, Landis MJ, Ronquist F, Huelsenbeck JP. 2014. Probabilistic graphical model representation in phylogenetics. Systematic Biology 63:753–771.

Höhna S, Landis MJ, Heath TA, Boussau B, Lartillot N, Moore BR, Huelsenbeck JP, Ronquist F. 2016. RevBayes: Bayesian phylogenetic inference using graphical models and an interactive model-specification language. Systematic Biology 65:726–736.

Huelsenbeck JP, Ronquist F. 2001. MRBAYES: Bayesian inference of phylogenetic trees. Bioinformatics 17:754–755.

Ji X, Fisher AA, Su S, Thorne JL, Potter B, Lemey P, Baele G, Suchard MA. 2023. Scalable Bayesian divergence time estimation with ratio transformations. Systematic Biology 72:1136–1153.

Ji X, Zhang Z, Holbrook A, Nishimura A, Baele G, Rambaut A, Lemey P, Suchard MA. 2020. Gradients do grow on trees: a linear-time O(N)-dimensional gradient for statistical phylogenetics. Molecular Biology and Evolution 37:3047–3060.

Jordan MI, Ghahramani Z, Jaakkola TS, Saul LK. 1999. An introduction to variational methods for graphical models. Machine Learning 37:183–233.

Jukes TH, Cantor CR. 1969. Evolution of protein molecules. Mammalian protein metabolism 3:21–132.

Kingma DP, Salimans T, Jozefowicz R, Chen X, Sutskever I, Welling M. 2016. Improved variational inference with inverse autoregressive flows. Advances in NSural Information Processing systems 29.

Kishino H, Thorne JL, Bruno WJ. 2001. Performance of a divergence time estimation

912 method under a probabilistic model of rate evolution. Molecular Biology and Evolution
913 18:352–361.

914 Koptagel H, Kviman O, Melin H, Safinianaini N, Lagergren J. 2022. VaiPhy: A variational
915 inference based algorithm for phylogeny. In: Advances in Neural Information Processing
916 Systems. volume 35. p. 14758–14770.

917 Kucukelbir A, Tran D, Ranganath R, Gelman A, Blei DM. 2017. Automatic differentiation
918 variational inference. The Journal of Machine Learning Research 18:430–474.

919 Kuhner MK, Yamato J, Felsenstein J. 1995. Estimating effective population size and
920 mutation rate from sequence data using metropolis-hastings sampling. Genetics
921 140:1421–1430.

922 Kumar R, Carroll C, Hartikainen A, Martin O. 2019. ArviZ a unified library for
923 exploratory analysis of bayesian models in python. Journal of Open Source Software
924 4:1143. doi:10.21105/joss.01143.

925 Lakner C, van der Mark P, Huelsenbeck JP, Larget B, Ronquist F. 2008. Efficiency of
926 Markov chain Monte Carlo tree proposals in Bayesian phylogenetics. Systematic Biology
927 57:86–103.

928 Lartillot N, Lepage T, Blanquart S. 2009. PhyloBayes 3: a Bayesian software package for
929 phylogenetic reconstruction and molecular dating. Bioinformatics 25:2286–2288.

930 Lunn DJ, Thomas A, Best N, Spiegelhalter D. 2000. WinBUGS - a Bayesian modelling
931 framework: concepts, structure, and extensibility. Statistics and Computing 10:325–337.

932 MacKay DJ. 2003. Information theory, inference and learning algorithms. Cambridge
933 university press.

934 Mateiu L, Rannala B. 2006. Inferring complex dna substitution processes on phylogenies
935 using uniformization and data augmentation. Systematic biology 55:259–269.

Matsen E, Rich DH, Milanov O, Fourment M, Jun SH, Nassif H, Kooperberg A, Kiami S, Ganapathy T, Yang L. 2023. `bito`.

Metropolis N, Rosenbluth AW, Rosenbluth MN, Teller AH, Teller E. 1953. Equation of state calculations by fast computing machines. The Journal of Chemical Physics 21:1087–1092.

Minin VN, Bloomquist EW, Suchard MA. 2008. Smooth skyride through a rough skyline: Bayesian coalescent-based inference of population dynamics. Molecular Biology and Evolution 25:1459–1471.

Paszke A, Gross S, Massa F, Lerer A, Bradbury J, Chanan G, Killeen T, Lin Z, Gimelshein N, Antiga L, Desmaison A, Köpf A, Yang E, DeVito Z, Raison M, Tejani A, Chilamkurthy S, Steiner B, Fang L, Bai J, Chintala S. 2019. PyTorch: an imperative style, high-performance deep learning library. Red Hook, NY, USA: Curran Associates Inc.

Piponi D, Moore D, Dillon JV. 2020. Joint distributions for TensorFlow Probability. arXiv preprint arXiv:2001.11819 .

Rezende D, Mohamed S. 2015. Variational inference with normalizing flows. In: F Bach, D Blei, editors. Proceedings of the 32nd International Conference on Machine Learning. volume 37. Lille, France: PMLR. p. 1530–1538.

Robbins H, Monro S. 1951. A stochastic approximation method. The Annals of Mathematical Statistics :400–407.

Ronquist F, Kudlicka J, Senderov V, Borgström J, Lartillot N, Lundén D, Murray L, Schön TB, Broman D. 2021. Universal probabilistic programming offers a powerful approach to statistical phylogenetics. Communications Biology 4:1–10.

Salvatier J, Wiecki TV, Fonnesbeck C. 2016. Probabilistic programming in python using PyMC3. PeerJ Computer Science 2:e55.

961 Senderov V, Kudlicka J, Lundén D, Palmkvist V, Braga MP, Granqvist E, Çaylak G,
962 Virgoulay T, Broman D, Ronquist F. 2024. TreePPL: A universal probabilistic
963 programming language for phylogenetics. bioRxiv doi:10.1101/2023.10.10.561673.

964 Stadler T. 2009. On incomplete sampling under birth–death models and connections to the
965 sampling-based coalescent. Journal of theoretical biology 261:58–66.

966 Stamatakis A. 2014. RAxML version 8: a tool for phylogenetic analysis and post-analysis
967 of large phylogenies. Bioinformatics 30:1312–1313.

968 Suchard MA, Rambaut A. 2009. Many-core algorithms for statistical phylogenetics.
969 Bioinformatics 25:1370–1376.

970 Swanepoel C. 2023. `phylojax`.

971 Tavaré S. 1986. Some probabilistic and statistical problems in the analysis of DNA
972 sequences. Lectures on Mathematics in the Life Sciences 17:57–86.

973 Thorne JL, Kishino H, Painter IS. 1998. Estimating the rate of evolution of the rate of
974 molecular evolution. Molecular Biology and Evolution 15:1647–1657.

975 To TH, Jung M, Lycett S, Gascuel O. 2016. Fast dating using least-squares criteria and
976 algorithms. Systematic Biology 65:82–97.

977 Vaughan TG, Kühnert D, Popinga A, Welch D, Drummond AJ. 2014. Efficient Bayesian
978 inference under the structured coalescent. Bioinformatics 30:2272–2279.

979 Xie W, Lewis PO, Fan Y, Kuo L, Chen MH. 2011. Improving marginal likelihood
980 estimation for Bayesian phylogenetic model selection. Systematic Biology 60:150–160.

981 Yang Z. 1994. Maximum likelihood phylogenetic estimation from DNA sequences with
982 variable rates over sites: approximate methods. Journal of Molecular Evolution
983 39:306–314.

Yang Z. 2007. PAML 4: phylogenetic analysis by maximum likelihood. Molecular Biology and Evolution 24:1586–1591.

Yang Z, Nielsen R, Goldman N, Pedersen AMK. 2000. Codon-substitution models for heterogeneous selection pressure at amino acid sites. Genetics 155:431–449.

Yang Z, Yoder AD. 1999. Estimation of the transition/transversion rate bias and species sampling. Journal of Molecular Evolution 48:274–283.

Yu Y, Abadi M, Barham P, Brevdo E, Burrows M, Davis A, Dean J, Ghemawat S, Harley T, Hawkins P, Isard M, Kudlur M, Monga R, Murray D, Zheng X. 2018. Dynamic control flow in large-scale machine learning. In: Proceedings of the Thirteenth EuroSys Conference. EuroSys '18. New York, NY, USA: Association for Computing Machinery. doi:10.1145/3190508.3190551.

Zhang C. 2020. Improved variational Bayesian phylogenetic inference with normalizing flows. Advances in Neural Information Processing Systems 33:18760–18771.

Zhang C, IV FAM. 2024. A variational approach to bayesian phylogenetic inference. Journal of Machine Learning Research 25:1–56.

Zhang C, Matsen IV FA. 2018a. Generalizing tree probability estimation via bayesian networks. Advances in Neural Information Processing Systems 31:1444–1453.

Zhang C, Matsen IV FA. 2018b. Variational Bayesian phylogenetic inference. In: International Conference on Learning Representations.

Zhang R, Drummond A. 2020. Improving the performance of Bayesian phylogenetic inference under relaxed clock models. BMC Evolutionary Biology 20:1–28.